

Lab Assignment

C++ Templates: Array, Optional, UniquePtr

1. Objective

Implement three template classes that mirror parts of the C++ standard library: **Array<T>** (a fixed-size heap-allocated array), **Optional<T>** (an optional value wrapper using in-place storage), and **UniquePtr<T, Deleter>** (an owning smart pointer with custom deleter support).

2. Array<T>

A generic fixed-size array allocated on the heap. The size is set at construction and cannot change.

2.1. Constructors and Destructor

```
Array(); // empty array, size 0
explicit Array(size_t size); // default-constructed elements
Array(size_t size, const T& value); // all elements = value

Array(const Array& other); // deep copy
Array(Array&& other); // move
Array& operator=(const Array& other); // copy assignment
Array& operator=(Array&& other); // move assignment

~Array();
```

2.2. Element Access

```
T& operator[](size_t index);
const T& operator[](size_t index) const;

T& front();
const T& front() const;

T& back();
const T& back() const;

T* data();
const T* data() const;
```

2.3. Capacity

```
size_t size() const;
bool empty() const;
```

2.4. Operations

```
void fill(const T& value);
void swap(Array& other);
```

2.5. Comparison

```
bool operator==(const Array& rhs) const;
```

```
bool operator!=(const Array& rhs) const;
```

Two arrays are equal if they have the same size and all corresponding elements are equal.

3. Optional<T>

A wrapper that may or may not contain a value of type T. The value is stored in-place using aligned storage -- no heap allocation is allowed. Use placement new to construct and explicit destructor calls to destroy the stored value.

3.1. Constructors and Destructor

```
Optional(); // empty (no value)
Optional(const T& value);
Optional(T&& value);

Optional(const Optional& other);
Optional(Optional&& other);
Optional& operator=(const Optional& other);
Optional& operator=(Optional&& other);
Optional& operator=(const T& value);
Optional& operator=(T&& value);

~Optional();
```

3.2. Observers

```
bool has_value() const;
explicit operator bool() const;

T& value();
const T& value() const;

T value_or(const T& default_value) const;

T& operator*();
const T& operator*() const;

T* operator->();
const T* operator->() const;
```

Calling `value()`, `operator*()`, or `operator->()` on an empty `Optional` is undefined behavior.

3.3. Modifiers

```
template <typename... Args>
T& emplace(Args&&... args); // destroy current (if any),
                           // construct new in-place

void reset(); // destroy value if present
void swap(Optional& other);
```

3.4. Comparison

```
bool operator==(const Optional& rhs) const;
bool operator!=(const Optional& rhs) const;
```

Two empty `Optionals` are equal. Two filled `Optionals` are equal if their values are equal. An empty `Optional` is never equal to a filled one.

3.5. Internal Storage

```
alignas(T) unsigned char storage_[sizeof(T)];
bool has_value_ = false;
```

You must use placement new to construct objects in `storage_` and explicitly call the destructor when destroying them. Do not use `new/delete`.

4. UniquePtr<T, Deleter>

An owning smart pointer that manages a single dynamically allocated object. It cannot be copied, only moved. Uses a customizable deleter (default: `std::default_delete<T>`).

4.1. Constructors and Destructor

```
UniquePtr(); // empty (nullptr)
explicit UniquePtr(T* ptr);
UniquePtr(T* ptr, const Deleter& deleter);
UniquePtr(T* ptr, Deleter&& deleter);

UniquePtr(const UniquePtr&) = delete;
UniquePtr& operator=(const UniquePtr&) = delete;

UniquePtr(UniquePtr&& other);
UniquePtr& operator=(UniquePtr&& other);

~UniquePtr();
```

The destructor must call the deleter on the managed pointer (if not null).

4.2. Observers

```
T*      get();
const T* get() const;

explicit operator bool() const;

T&      operator*();
const T& operator*() const;

T*      operator->();
const T* operator->() const;

Deleter& get_deleter();
const Deleter& get_deleter() const;
```

4.3. Modifiers

```
T*      release(); // give up ownership
void reset(T* ptr = nullptr); // delete current, take new
void swap(UniquePtr& other);
```

`release()` returns the raw pointer and sets the internal pointer to null without calling the deleter. `reset()` calls the deleter on the current pointer (if not null), then takes ownership of the new pointer.

4.4. Free Function

```
template <typename T, typename... Args>
UniquePtr<T> make_unique(Args&&... args);
```

Allocates and constructs a T with the given arguments, returning a `UniquePtr<T>` that owns it. Uses perfect forwarding.

4.5. Array Specialization: UniquePtr<T[]>

A partial specialization for dynamically allocated arrays. The default deleter is `std::default_delete<T[]>` (calls `delete[]`). Provides `operator[]` instead of `operator*/operator->`.

```

template <typename T, typename Deleter>
class UniquePtr<T[], Deleter> {
public:
    UniquePtr();
    explicit UniquePtr(T* ptr);
    UniquePtr(T* ptr, const Deleter& deleter);
    UniquePtr(T* ptr, Deleter&& deleter);

    UniquePtr(const UniquePtr&) = delete;
    UniquePtr& operator=(const UniquePtr&) = delete;

    UniquePtr(UniquePtr&& other);
    UniquePtr& operator=(UniquePtr&& other);

    ~UniquePtr();

    T*      get();
    const T* get() const;
    explicit operator bool() const;

    T&      operator[](size_t index);      // instead of operator*
    const T& operator[](size_t index) const;

    Deleter&      get_deleter();
    const Deleter& get_deleter() const;

    T*  release();
    void reset(T* ptr = nullptr);
    void swap(UniquePtr& other);
};

```

5. Implementation Requirements

- All implementations go in the header files (.h), below the class declarations.
- You **must not** modify the public interface of the classes.
- You **may** modify the private sections (e.g. member attributes, helper methods).
- You **must not** modify test files or tracked.h.
- The code must compile with the C++17 standard.
- No third-party libraries allowed.
- `Optional<T>` must not use heap allocation. Store the value in-place using `aligned_storage` / `placement new`.

6. Compiler Flags and Sanitizers

```

-Wall -Wextra -Wpedantic -Werror
-fsanitize=address,undefined -fno-sanitize-recover=all

```

All compiler warnings are treated as errors. Your code must compile cleanly with zero warnings.

AddressSanitizer (ASan) detects memory errors: buffer overflows, use-after-free, memory leaks, and similar issues. UndefinedBehaviorSanitizer (UBSan) detects undefined behavior: signed integer overflow, null pointer dereferences, misaligned access, etc. If any sanitizer error is triggered during testing, the test will fail.